



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

COMPLEX PROBLEMS

AP LAB-2

Student Name: Karanvir Singh

UID: 22BCS16269

Section/Group: NTPP 602-A

Date :07-04-25

Q1 .

Trapping Rain Water

Objective

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining

Algorithm

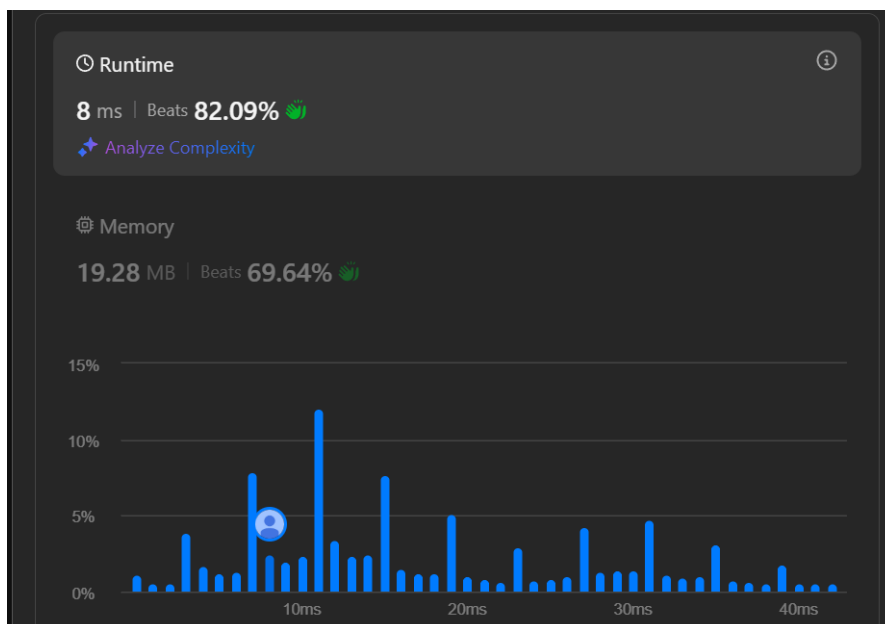
- Initialize `i`, `j`, `left_max`, `right_max`, and `sum`.
- Iterate while `i < j`.
- Compare `left_max` and `right_max` to decide whether to process the left or right side.
- Calculate trapped water and update `i` or `j`, and adjust `left_max` or `right_max`.
- Return the total trapped water `sum`.

Implemetation/Code

class Solution:

```
def trap(self, height: List[int]) -> int:  
    i = 0  
    left_max = height[0]  
    sum = 0
```

```
j = len(height) - 1
right_max = height[j]
while i < j:
    if left_max <= right_max:
        sum += left_max - height[i]
        i += 1
        left_max = max(left_max, height[i])
    else:
        sum += right_max - height[j]
        j -= 1
        right_max = max(right_max, height[j])
return sum
```



Time Complexity : $O(n)$

Space Complexity : $O(1)$



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Q2 .

Word Break

Objective

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Algorithm

- Convert `wordDict` into a set (`wordSet`) for faster lookup.
- Define a helper function `construct(current)` that uses recursion and memoization to check if the string `current` can be broken down into words from `wordDict`.
- If the string is empty (`current == ""`), return `True`.
- Each word in `wordSet`, check if it is a prefix of `current` and recursively check remainder of the string.
- Use memoization to store previously computed results for subproblems, returning the result of `construct(s)` at the end.

Implementation/Code

```
import re
from itertools import accumulate
```

class Solution:

```
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        wordSet = set(wordDict)
        memo = {}
        def construct(current):
            if current in memo: return memo[current]
            if not current: return True
            memo[current] = any(current.startswith(word) and construct(current[len(word):]) for
word in wordSet)
            return memo[current]
        return construct(s)
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Q3.

Longest Substring Without Repeating Characters

Objective

Given a string s , find the length of the longest substring without duplicate characters.

Algorithm

- Initialize `seen`, `l`, and `output`.
- Iterate through the string with a right pointer r .
- If $s[r]$ is in the window, move `l` to the right of its last occurrence.
- Update `seen[s[r]]` with the current index.
- Update `output` with the maximum window size ($r - l + 1$).

Implementation/Code

class Solution:

```
def lengthOfLongestSubstring(self, s: str) -> int:
```

```
    seen = {}
```

```
    l = 0
```

```
    output = 0
```

```
    for r in range(len(s)):
```

```
        if s[r] in seen and seen[s[r]] >= l:
```

```
            l = seen[s[r]] + 1
```

```
        seen[s[r]] = r
```

```
        output = max(output, r - l + 1)
```

```
    return output
```

Time Complexity : $O(1)$

Space Complexity : $O(\min(n,m))$